

Partie 09 — Exercices

Fiabiliser et partager une analyse

Oskar Laverny

Table des matières

Exercice 1 — Que faut-il transmettre ?	1
Exercice 2 — Une dépendance cachée	1
Exercice 3 — Décrire l’environnement	1
Exercice 4 — Initialiser son environnement avec <code>renv</code>	2
Exercice 5 — Une fonction et son contrat	2
Exercice 6 — Réparer une analyse fragile	3
Exercice 7 — Partager sans oublier l’environnement	3
Exercice 8 — Un environnement qui a changé	3
Exercice 9 — Reproductible, mais faux	3

Les exercices sont autonomes : aucun ancien devoir n’est nécessaire. Le socle est constitué des exercices 1 à 5. L’exercice 4 nécessite un accès Internet pour installer les packages ; travaillez en binôme si nécessaire. Les exercices 6 à 9 servent de réserve. Les commandes de rendu s’exécutent dans le terminal ; les commandes R dans R.

Exercice 1 — Que faut-il transmettre ?

Un collègue vous envoie seulement `rapport.pdf`. Son analyse utilise `rapport.qmd`, `R/fonctions.R` et `data/patients.csv`, ainsi que des packages R. Quels fichiers et informations lui demander pour refaire les calculs ? Expliquez la différence entre une source et une sortie générée. Pourquoi un chemin comme `C:/Users/auteur/Documents/patients.csv` ne convient-il pas pour transmettre le projet ? Proposez un chemin relatif.

Exercice 2 — Une dépendance cachée

Un script `analyse.R` contient uniquement :

```
tas <- c(120, 145, NA, 160)
mean(tas >= seuil, na.rm = TRUE)
```

Son auteur avait défini `seuil <- 140` dans sa console. Pourquoi ce script échoue-t-il dans une nouvelle session ? Rendez-le autonome, puis testez un seuil de 150. Expliquez la règle utilisée pour les valeurs manquantes.

Exercice 3 — Décrire l’environnement

Exécutez `sessionInfo()` dans votre session. Relevez la version de R et, si vous en avez un, celle d’un package chargé. Quelle est la différence entre installer et charger un package ? Une

capture de `sessionInfo()` réinstalle-t-elle les packages ? Retrouvez également la version de Quarto avec `quarto --version`.

Exercice 4 — Initialiser son environnement avec renv

Dans une session donnée, par exemple celle du dossier du cours.

1. installez `renv` si nécessaire, puis initialisez le projet et observez la bibliothèque active :

```
install.packages("renv") # seulement si nécessaire
renv::init()
.libPaths()
```

2. Créez et enregistrez `analyse.R` avec le code suivant. Installez `digest` dans la bibliothèque du projet avec `install.packages("digest")`, puis exécutez le script :

```
tas <- c(120, 145, NA, 160)
print(mean(tas, na.rm = TRUE))
print(digest::digest(tas))
```

La deuxième ligne affichée est une empreinte du vecteur ; elle sert ici à utiliser explicitement un petit package, sans dépendance graphique.

3. Exécutez `renv::snapshot()`, puis `renv::status()` ; acceptez les confirmations après lecture. Ouvrez `renv.lock` et retrouvez la version de `digest`. Identifiez aussi `.Rprofile` et `renv/activate.R` (fichiers parfois cachés).
4. Fermez puis rouvrez le projet. Vérifiez `.libPaths()`, relancez le script et exécutez `renv::restore()`. Pourquoi cette restauration peut-elle n'avoir aucun package à réinstaller ?
5. Expliquez la différence entre `init()`, `snapshot()`, `restore()` et `status()`. Quelle commande utiliser après l'ajout d'une dépendance et un test réussi ? Pourquoi `renv` ne suffit-il pas à installer R et Quarto ?
6. À la fin de l'essai, utilisez `renv::deactivate()` pour retrouver l'environnement habituel. Redémarrez R si nécessaire, puis vérifiez `.libPaths()`. Le verrouillage et la bibliothèque du projet sont conservés par défaut ; n'utilisez pas `clean = TRUE`. Pour réactiver ultérieurement cet environnement : `renv::activate()`.

Exercice 5 — Une fonction et son contrat

Écrivez `proportion_seuil(x, seuil = 140)` : `x` doit être numérique, le seuil doit être un nombre unique et fini. Retirez les valeurs manquantes. Sans observation disponible, arrêtez le calcul avec un message explicite.

Testez la fonction avec `c(120, 145, NA, 160)` aux seuils 140 et 150. Ajoutez des tests avec `stopifnot()` et `isTRUE(all.equal())`. Vérifiez les cas : toutes les valeurs sous le seuil, toutes au-dessus, une valeur exactement au seuil. Observez séparément les erreurs pour un vecteur vide, uniquement des valeurs manquantes et du texte.

Exercice 6 — Réparer une analyse fragile

Un auteur exécute d'abord `seuil <- 140` dans sa console, puis utilise ces cellules dans son rapport :

```
patients <- read.csv("C:/Users/auteur/Documents/patients.csv")
proportion <- sum(patients$tas >= seuil, na.rm = TRUE) / nrow(patients)
moyennes <- replicate(1000, mean(sample(patients$tas, 3, replace = TRUE)))
```

Le CSV comporte la colonne `tas` et les quatre valeurs 120, 145, NA, 160. Identifiez au moins quatre fragilités. Réécrivez le code en utilisant `data/patients.csv`, une proportion parmi les observations disponibles et une simulation sur les valeurs observées, avec une graine fixée. Vérifiez la proportion attendue et expliquez la population simulée.

Exercice 7 — Partager sans oublier l'environnement

Préparez un README pour le projet corrigé : données, règles de calcul, prérequis, restauration `renv`, commande de rendu et résultat attendu. Listez les fichiers à transmettre, y compris les fichiers cachés d'activation, et ceux à exclure. Échangez un dossier avec un voisin et tentez un rendu depuis une nouvelle session. Notez les informations manquantes.

Exercice 8 — Un environnement qui a changé

Un projet reçu attend une ancienne version d'un package dans `renv.lock`. La bibliothèque locale contient une autre version. Faut-il commencer par `snapshot()` ou `restore()` ? Quel est le rôle de `status()` ? L'auteur ajoute ensuite un package et vérifie son analyse : quelle commande utilise-t-il avant de transmettre le projet à nouveau ? Pourquoi un verrouillage ne garantit-il pas que R et Quarto sont installés ?

Exercice 9 — Reproductible, mais faux

Un programme calcule `sum(x, na.rm = TRUE) / length(x)` pour estimer la moyenne des valeurs observées. Il fixe sa graine et possède un `renv.lock`. Est-il pour autant correct ? Construisez un petit cas qui révèle le problème, corrigez la formule et testez-la face à `mean(x, na.rm = TRUE)`. Discutez le cas où toutes les valeurs sont manquantes. Expliquez ce qu'un test réussi ne démontre pas.